

Bug Management

Pengurusan Pepijat & Metrik Kualiti

Module 6 of 9, tracking, classifying and resolving defects with discipline, then proving quality with numbers instead of opinions.

TIME

10:45-12:30

DURATION

1h 45m

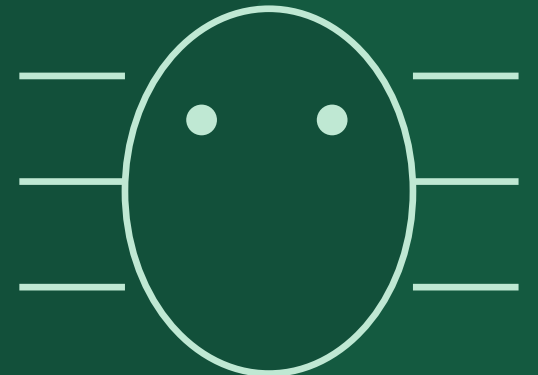
PHASE

Verify

PROJECT

ShopFast

KRISA 2.0 · Ch.5 Development, System Test Report



Jadual slot: Modul 6: Pengurusan Pepijat dan Metrik Kualiti

Run sheet / session timing

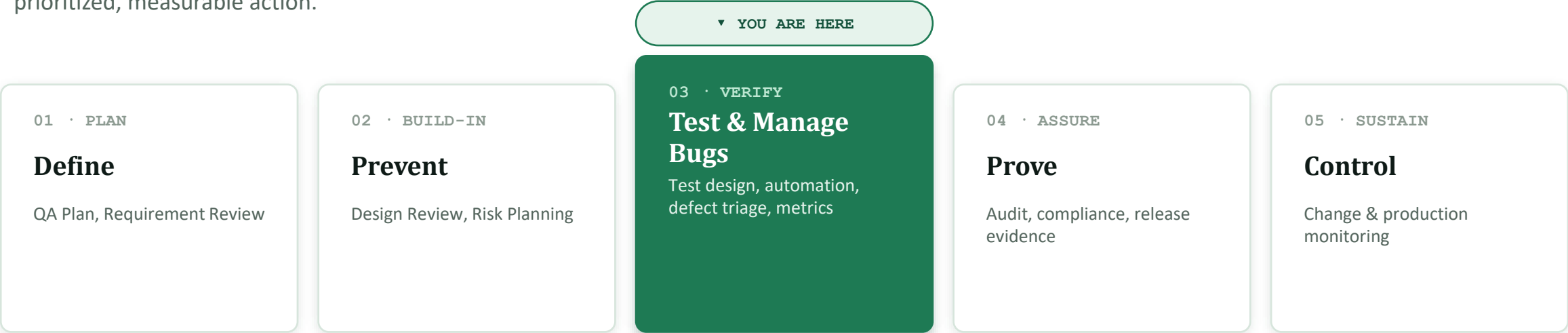
Hari 2, 10:45 hingga 12:30 · 105 minit · kira-kira 80% amali



Masa	Segmen	Tempoh
10:45 hingga 11:10	Konsep	25 minit
11:10 hingga 12:00	Makmal	50 minit
12:00 hingga 12:25	Bengkel AI W06	25 minit
12:25 hingga 12:30	Rumusan	5 minit

Where Module 6 sits in the quality flow

Nine SQA activities map onto five delivery phases. Bug Management is the engine room of Verify, where every test result becomes a tracked, prioritized, measurable action.



Follows: **Module 5, Software Testing & Test Automation**
Feeds into: **Module 7, Audit, Compliance & Documentation**



Track, classify and manage defects **effectively**, and turn testing activity into **meaningful quality metrics**.

Reproduce with evidence

Write bug reports that don't bounce
back

Triage by severity & priority

Measure instead of guess

Six concepts, one shared vocabulary

Every topic gets a hands-on touchpoint in the lab, the language your team needs to triage a bug in under two minutes.

1

Defect Lifecycle

The states a bug moves through, from first sighting to closed, and who owns each handoff.

2

Severity vs Priority

How bad it is, versus how soon it must be fixed, two independent axes, often confused.

3

Root Cause Analysis

Asking “why” until you reach the actual source, not just the symptom reported.

4

Defect Density & Leakage

How many bugs per unit of work, and how many escape into later stages.

5

MTTR & MTBF

Mean time to repair, and mean time between failures, speed and stability.

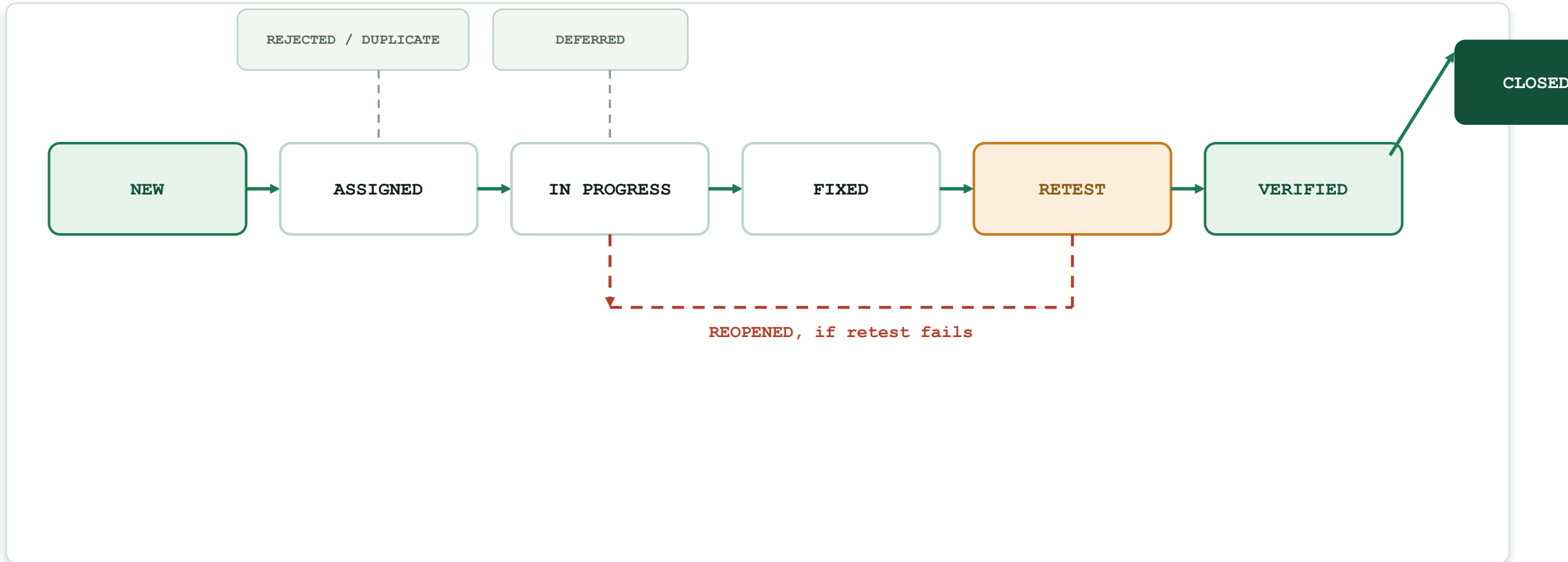
6

Defect Removal Efficiency

The percentage of defects caught before release vs. found after, the scorecard.

The defect lifecycle

A bug moves through a governed state machine. Miss a state and you lose traceability.

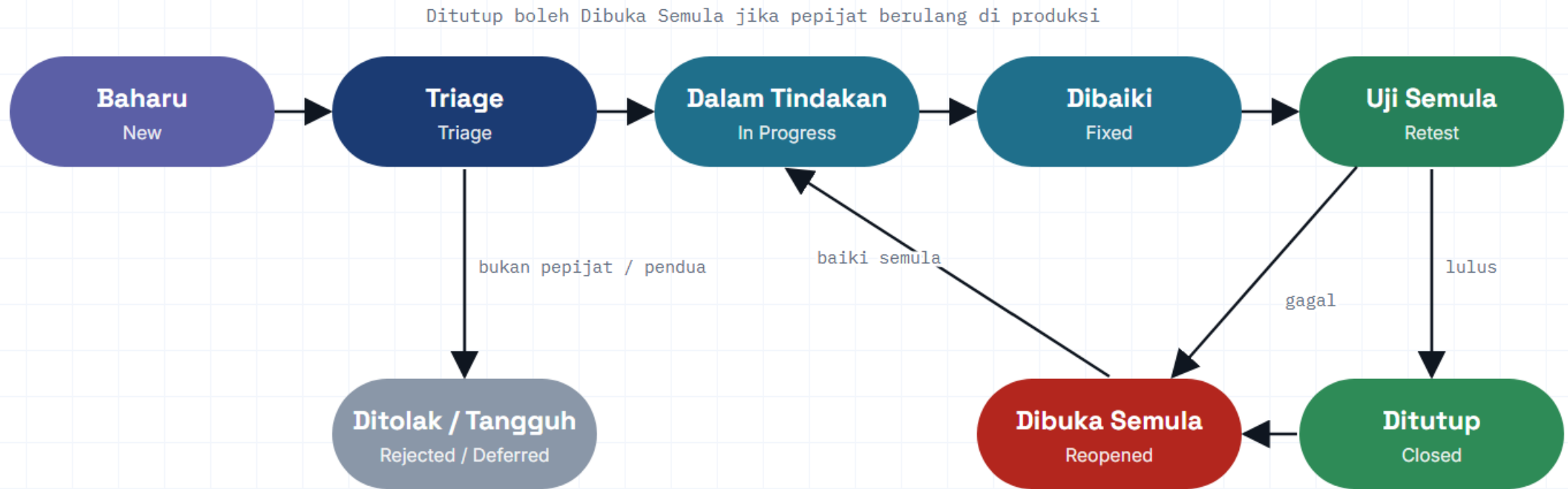


Lab focus: reproduce a seeded bug, push it through New to Assigned to In Progress to Fixed to Retest to Verified to Closed, and practice the reopen path when a fix doesn't hold.

Kitar hayat pepijat (defect lifecycle)

Defect lifecycle

KRISAv2 Bab 6 6.3 Pengurusan Ralat: daftar ralat, tahap severity dan keutamaan, uji semula, triage oleh PM, Ketua Ujian dan Ketua Pembangun



Kriteria keluar UAT (Bab 6 6.8): 100% senario dilaksana, sekurang-kurangnya 95% kes lulus, semua ralat severity 1 dan 2 ditutup.

Severity vs. priority

Severity is a fact about impact. Priority is a decision about sequence.

PRIORITY to



LOW

HIGH

■ Severity

Technical impact on the system, crash, data loss, blocked workflow. Set by QA based on observed behavior.

■ Priority

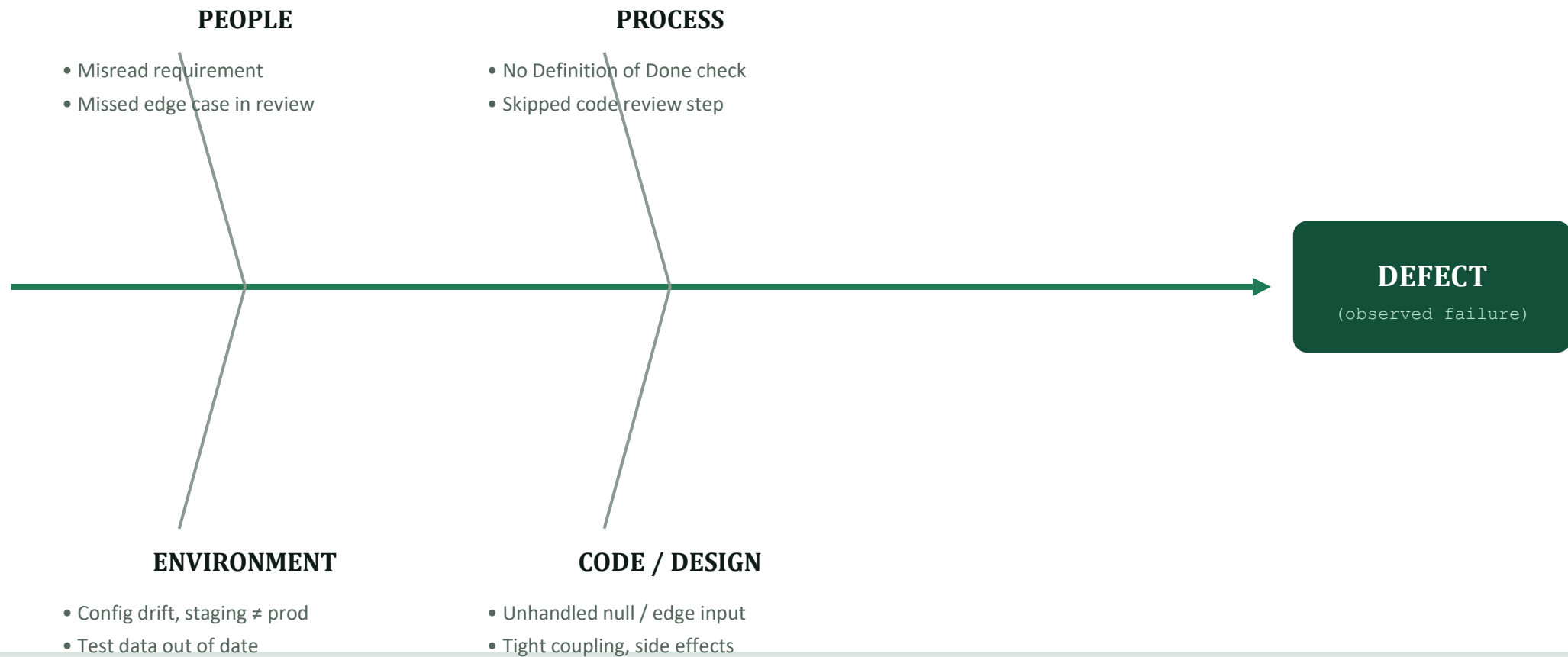
Business urgency, how soon it must be fixed relative to everything else. Set by Product/PM.

■ Lab rule of thumb

Log both independently for every ShopFast bug, never infer one from the other.

Root Cause Analysis, fishbone view

Trace every bug back to a category of cause before closing it, this keeps the same defect from resurfacing.



Lab focus: for one logged ShopFast bug, ask “why” three times and record which of these four categories it truly belongs to.

Turning bug data into a quality story

Five metrics, five formulas, computed by hand once in the lab so the dashboard never feels like a black box.

Defect Density

Defects ÷ KLOC

0.42

Bugs per unit of code size,
lower is tighter code.

Defect Leakage

Escaped ÷ Total × 100

12%

Share of bugs that slipped
past a test stage.

MTTR

$\Sigma \text{ repair time} \div \#$
incidents

3.1h

Mean time to repair, how fast
the team recovers.

MTBF

Total uptime ÷ #
failures

96h

Mean time between failures,
system stability.

DRE

Pre ÷ (pre + post) ×
100

88%

Defect Removal Efficiency, the
release scorecard.

Bug Management and Metrics

How reliability thinking, defect tracking and measurement turn test results into decisions

01 Software reliability and operational profiles

02 Removing faults vs improving reliability

03 A bug's life: states and roles

04 Software measurement and its process

05 Control vs predictor, static vs dynamic metrics

06 Theilities, code metrics and OO metrics

07 Defect density and defect clustering

08 Defect prevention practices

What is software reliability?

Software reliability

The probability of failure-free operation for a specified time in a specified environment. Informally, it is how well the software provides the services users need, as the users themselves perceive it.

Not an objective property of the code alone: it depends on how the software is actually used.

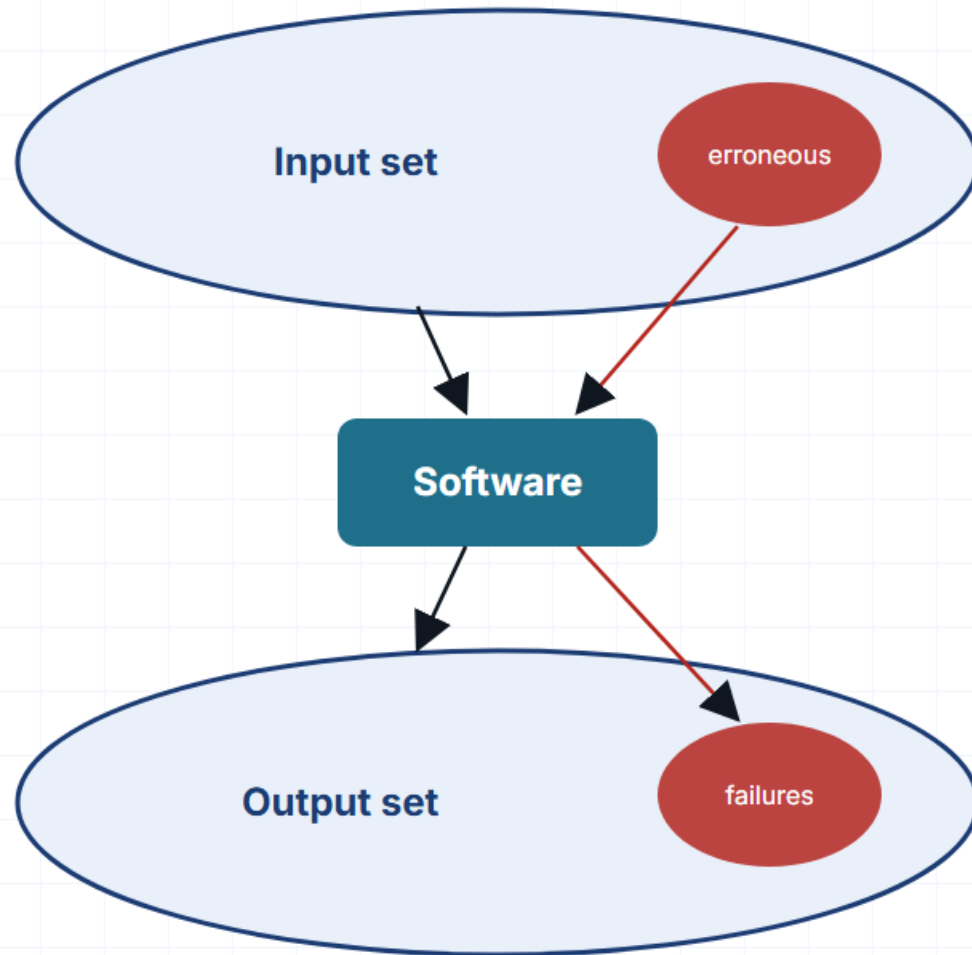
Must be based on an operational profile: the expected pattern of inputs from real users.

Must weigh consequences: a wrong order total is far worse than a misaligned icon.

ShopFast: time could be 30 days of peak sales; environment the production web tier.

Reliability depends on how the software is used

Failures only appear for the inputs that users actually send



Reliability: probability of failure-free operation for a specified time in a specified environment.

Operational profile: which inputs users really send. A member buying one Songkok never touches the MEGA20 cap rule, a festive-sale shopper does.

Fault vs failure: a fault in rarely used code may never cause a failure for most users, so reliability is measured from the user's view.

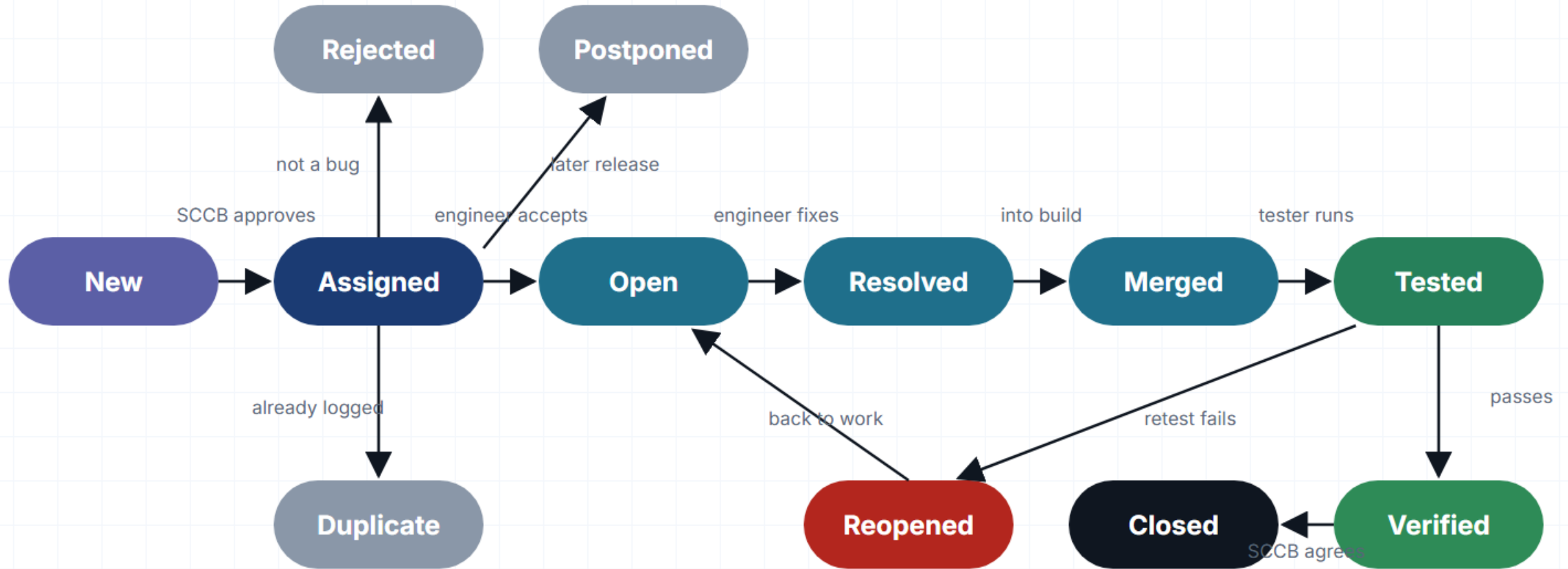
Removing faults is not the same as improving reliability

Where a fault sits matters more than how many faults are removed

- **The finding:** Mills et al. (1987), as cited in the course notes: removing 60 percent of faults gave only a 3 percent improvement in reliability.
- **Why:** Many faults sit in rarely executed code. Removing them changes little of what users actually experience.
- **Priority 1: frequency:** Reliability improves most when faults in the most frequently used parts of the software are removed.
- **Priority 2: consequence:** Removing faults with the most serious consequences is the primary objective, even when they are rare.
- **ShopFast triage:** A wrong order total for quantities 1 to 10 outranks a typo in a yearly admin report, although each is one bug.
- **Practical tool:** Use the operational profile (usage logs, analytics) to decide where to test harder and which fixes go first.

A bug's life: states and who moves it

Every transition has an owner, so a report never gets lost



SCCB = software configuration control board. **STR** = software trouble report, the form that carries the bug through these states.

Bug states and who moves the bug

Based on the Raytheon STR workflow. SCCB is the software configuration control board

State	What it means	Who moves it on
New	STR raised with steps to reproduce, evidence and severity	SCCB reviews and approves the STR for work
Assigned	Approved STR given to an engineer	Engineer accepts the STR
Open	Engineer is analysing and fixing the problem	Engineer resolves the STR
Resolved	Fix made on the engineer's workspace or branch	Software lead plans the merge; integrator performs it
Merged	Fix is in the integrated build	Tester verifies the fix in that build
Tested	Fix verified by a tester	SCCB agrees closure
Verified / Closed	Closure agreed and recorded	Reopened only through the project's defined procedure
Rejected	Not a defect, or works as specified	SCCB at triage, with the reason recorded
Postponed	Valid, but deferred to a later release	SCCB, with a target release recorded
Duplicate	Same problem as an existing STR	SCCB links it to the original STR

What is software measurement?

Software measurement

The process of deriving a numeric value for some attribute of a software product or a software process. Comparing these values with each other and with standards lets us draw conclusions about quality.

Most metrics programmes focus first on defects and on the verification and validation process.

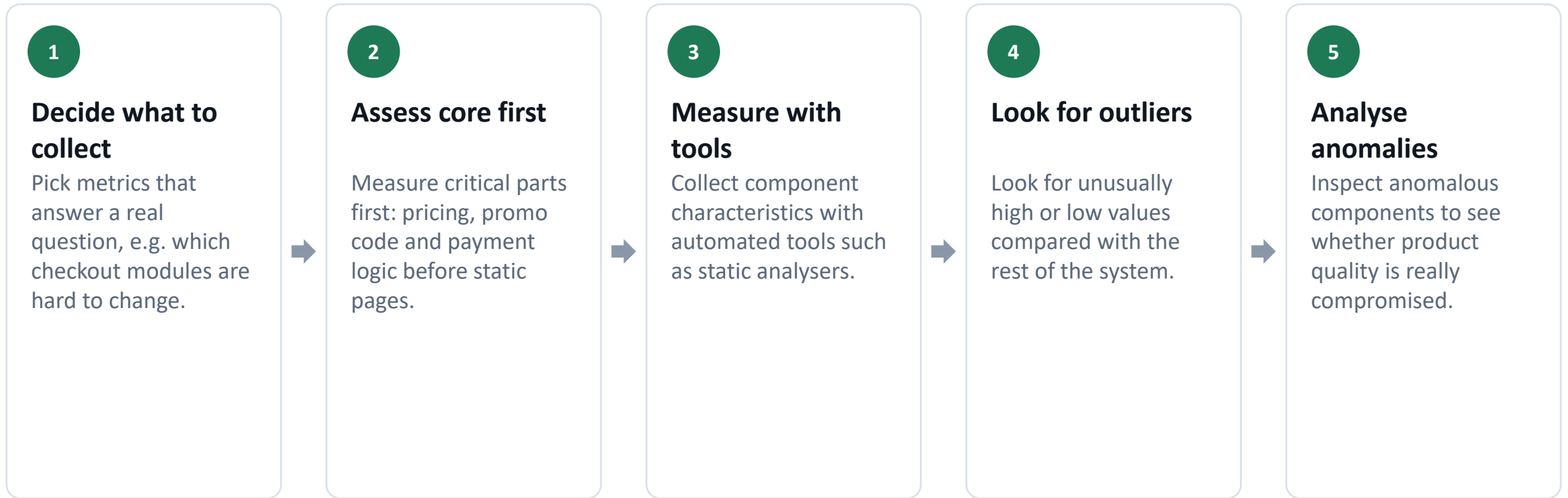
Set boundary values in advance, then revise them once enough historical data has been collected.

Metrics must be standardised: if teams count defects or lines differently, comparisons mislead.

ShopFast: open STRs by severity, defects found per review, test pass rate per build.

The process of product measurement

Five steps to measure components without drowning in numbers



An outlier is a prompt to look closer, not proof of poor quality. Some components are complex because the problem they solve is complex.

Control metrics vs predictor metrics

One kind steers the process, the other forecasts the product

Control metrics (process)

- Support process management: is the team working effectively?
- STR aging: many old, unresolved STRs signal low productivity or a blocked team
- Requirements volatility: a high change rate signals scope creep
- ShopFast: change requests to promo rules after the design baseline

Predictor metrics (product)

- Measure product attributes to forecast quality characteristics
- Nesting depth above 5 predicts low readability
- Any open priority 1 STR at SAT predicts low product reliability
- ShopFast: complexity of the shipping fee function predicts test effort

Use control metrics to manage how the team works and predictor metrics to judge what the team built. Both need agreed definitions and thresholds.

Static vs dynamic product metrics

The course notes name size (SLOC) and nesting depth as the most reliable static predictors of maintainability

Dynamic metrics

Collected while the program runs. Help assess efficiency and reliability. Easy to measure, e.g. execution time, mean time between failures.

Static metrics

Collected from the products of development: code, designs, documents. Help assess complexity, understandability and maintainability.

ShopFast dynamic examples

Checkout response time under load; failures per 1,000 orders in staging; mean time between payment errors.

ShopFast static examples

Lines of code and nesting depth of the order total function; number of parameters in the promo code API.

The ilities cannot be measured directly

Measure tangible internal attributes, then derive the quality factor through an agreed and validated relationship

Quality attribute	Measurable attributes used as indicators	Caution
Maintainability	Parameters per procedure, cyclomatic complexity, size (LOC), error messages, manual length	Calibrate the relationship with your own historical data
Reliability	Cyclomatic complexity, program size in LOC, number of error messages	Static indicators only; confirm with failure data from operation
Portability	Number of procedure parameters, program size in LOC	Platform-specific calls matter too, and size does not show them
Usability	Number of error messages, length of user manual	A long manual may mean poor usability or simply thorough help

Classic code metrics and what they signal

ShopFast: a shipping fee function with deep nesting and high fan-in is a review and test priority

Metric	What it measures	A high value suggests
Fan-in	Number of functions that call a given function X	X is tightly coupled; changes to X have wide knock-on effects
Fan-out	Number of functions that X calls	X has complex control logic to coordinate the called components
Length of code	Size of a program component, e.g. lines of code	Larger components tend to be more complex and error-prone
Cyclomatic complexity	Control complexity: independent paths through the code (McCabe)	Harder to understand, and more test cases are needed
Length of identifiers	Average length of distinct identifiers in a program	Names are more likely meaningful, so the program is easier to understand
Depth of conditional nesting	How deeply if-statements are nested	Code is hard to understand and potentially error-prone
Fog index	Average length of words and sentences in documents	The document is harder to read

Object-oriented metrics

OO designs need metrics that see classes and inheritance, not only functions

OO metric	What it measures	A high value suggests
Depth of inheritance tree	Number of levels where subclasses inherit attributes and operations	Complex design: many classes must be understood to grasp a leaf class
Method fan-in/fan-out	Calls into and out of a method; internal and external calls may be separated	Same as function fan-in/fan-out: coupling and control complexity
Weighted methods per class	Number of methods in a class, each weighted by its complexity	Complex class, harder to understand, may lack cohesion and reuse poorly
Number of overriding operations	Superclass operations overridden in a subclass	The superclass may not be an appropriate parent for the subclass

Defect density: a benchmark with a date on it

Defects per thousand lines of code (KLOC) is simple to compute and easy to misuse

- **Definition:** Defect density is confirmed defects divided by size in KLOC, for a stated phase or period such as the first six months in use.
- **Historical figure:** A Bell Labs survey, cited in Leveson's 1999 notes, found fielded software errors typically ranged from 0.5 to 3 per 1,000 lines of code.
- **Treat as history:** The figure predates modern languages, frameworks and CI. Use it to explain the idea, not as a target for your project.
- **Define before counting:** Agree what counts as a defect (severity, duplicates) and how KLOC is counted, or comparisons across projects mislead.
- **Normalise and trend:** Track your own density per release and per module. The trend tells you more than any single number.
- **ShopFast use:** Compare density across checkout modules to find weak spots, never to rank individual developers.

Defect clustering: what it means for triage

Myers: the chance of more errors in a section rises with the number of errors already found there

Bugs travel in groups

A module that already produced many STRs probably hides more. Clusters point to complex, unclear or rushed work.

Widen the retest

When a fix lands in a clustered module, extend regression tests around it instead of testing the fix alone.

Review before patching

Several STRs in one area justify an inspection or redesign rather than yet another patch.

Tag by component

Record the component on every STR so clusters appear in reports. Untagged bugs hide patterns.

ShopFast example

Five STRs in promo code handling in one sprint: schedule a code inspection and add boundary tests for codes and totals.

Do not overcorrect

A quiet module is not proven clean. Keep baseline coverage everywhere, including rarely tested areas.

Defect prevention practices

Lower the reliance on detection by stopping defects from being created

- ☐ Lessons learned: learn from other projects and share yours, e.g. a promo code post-mortem shared across teams
- ☐ Managing with metrics: collect standard metrics, understand them, and change product or process accordingly
- ☐ Risk analysis: identify risks and opportunities early and track each one until it is closed or realised
- ☐ Build freeze: no code changes on the build while formal tests such as FAT and SAT are running
- ☐ Unit-level testing guidelines: a test plan and procedure for each unit, e.g. quantity 0, 1, 10 and 11
- ☐ Baseline acceptance criteria: closure criteria agreed in advance, e.g. no priority 1 STRs at FAT readiness review
- ☐ Root cause for escaped defects: ask which earlier activity should have caught the defect, and fix that activity

Detection finds defects that already exist. Prevention changes the process so the same kind of defect is not created again.

Hands-on: from reproduction to metrics

PROJECT ShopFast checkout, the same project carried through every module of this workshop.

01

Reproduce

Reproduce seeded bugs in the demo app and confirm exact repro steps before logging anything.



02

Log 5 reports

Write 5 high-quality bug reports: steps, expected vs. actual, severity/priority, and evidence.



03

Triage the board

Move tickets across the lifecycle: Open to In Progress to Fixed to Verified to Closed.



04

Compute metrics

Calculate simple metrics from your own data: Defect Density & pass rate.

Real run: capture exact evidence before you log a bug

Request, status code, response body and build version from the ShopFast lab app (PowerShell)

```
SQA-SHOT
# Step 1: add an out-of-stock product (P006) and capture the exact response
PS> Invoke-WebRequest -Method Post http://localhost:3000/api/cart/items -Headers $h -ContentType application/json -Body '{"productId":"P006","quantity":1}'
StatusCode : 400
Body       : {"error":"OUT_OF_STOCK","message":"Dodol Durian Musang King does not have enough stock"}

# Step 2: same request with a product in stock, to prove the difference
PS> Invoke-RestMethod -Method Post http://localhost:3000/api/cart/items -Headers $h -ContentType application/json -Body '{"productId":"P002","quantity":2}'

subtotal : 70
discount  : 0
shipping  : 8
total     : 78

# Step 3: record the build under test for the bug report
PS> Invoke-RestMethod http://localhost:3000/api/health

status    : ok
version    : 1.0.0

PS C:\Users\user\sqa-krisa-bengkel\lab\shopfast>
```

What you'll use in the lab

Industry-standard and swappable, use whichever your organization already runs.

1

[Jira](#)

Bug tracking & workflow

2

GitHub Issues

Lightweight defect logging

3

Azure DevOps / GitLab

Enterprise triage boards

4

Bug report template

Consistent, reviewable reports

5

Metrics sheet

Density, leakage, MTTR/MTBF,
DRE

6

RTM (carried forward)

Linking bugs back to
requirements

Module 6 deliverables

5 defect reports

Payment fails on retry	HIGH
Discount code not applied	MED
Cart badge miscounts	MED
Footer link 404s	LOW
Confirmation email delay	LOW

Each with steps, expected vs. actual, severity/priority & evidence.

A moving triage board



Practiced end-to-end lifecycle movement, not just static logging.

Basic quality metrics

Defect Density	0.42 / KLOC
Pass Rate	82%
Defect Leakage	12%

Computed from your own ShopFast bug data, not a slide of theory.

Abbreviations used in this module (1 of 4)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
AI	Artificial Intelligence	Software that can answer questions and write text or code by learning patterns from large amounts of data.
API	Application Programming Interface	A doorway that lets one program ask another program for data or an action, without using a screen.
BVA	Boundary Value Analysis	Testing at the edges of the allowed values, such as 0, 1, 10 and 11, because mistakes often hide there.
CI	Continuous Integration, or Configuration Item	Two meanings. In DevOps, every code change is built and tested automatically. In configuration management, it is any controlled item such as a document or code file.
DRE	Defect Removal Efficiency	The share of all defects that were caught before release. Higher is better.
EP	Equivalence Partitioning	Grouping inputs that behave the same way, then testing one value from each group.
FAT	Factory Acceptance Test	The formal test with the customer before the system is installed at the live site.
ID	Identifier	A unique label, such as REQ-CHK-01, so that each item can be traced.

Abbreviations used in this module (2 of 4)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
JDN	Jabatan Digital Negara	The Malaysian government department that publishes KRISA and PPrISA.
KLOC	Thousand Lines of Code	A size measure used in rates such as defects per KLOC.
KRISA	Kejuruteraan Sistem Aplikasi Sektor Awam	The Malaysian public sector guide for engineering application systems. KRISAv2 is the 2026 beta edition.
LOC	Lines of Code	The number of program lines. A rough measure of size.
MIT	Massachusetts Institute of Technology	A university in the United States. Source of the Leveson course notes.
MTBF	Mean Time Between Failures	The average time a system runs before it fails. Higher is better.
MTTR	Mean Time To Repair	The average time needed to get a failed system working again. Lower is better.
OO	Object-Oriented	A way of writing programs as objects that hold both data and behaviour.

Abbreviations used in this module (3 of 4)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
PAT	Provisional Acceptance Test	The KRISA acceptance test done before provisional acceptance of the system, including performance checks.
PDF	Portable Document Format	A document file that looks the same on every computer.
PM	Product Manager or Project Manager	The person who sets the priority of what is built or fixed first.
PPrISA	Panduan Pengurusan Projek ICT Sektor Awam	The Malaysian public sector guide for managing ICT projects. Codes such as PPrISA05 are its templates.
PS	PowerShell prompt	When a slide shows PS>, type that command in Windows PowerShell.
QA	Quality Assurance	Preventing defects by setting up good processes and checking that people follow them.
REQ	Requirement	Prefix in requirement IDs such as REQ-CHK-01. CHK stands for checkout.
RTM	Requirements Traceability Matrix	A table linking each requirement to its design, tests and defects, so nothing is missed.

Abbreviations used in this module (4 of 4)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
SAT	Site Acceptance Test	The formal acceptance test carried out at the live site.
SCCB	Software Configuration Control Board	The board that controls changes to software baselines, including which bug fixes go in.
SLOC	Source Lines of Code	Lines of program code, not counting comments or blank lines.
SQA	Software Quality Assurance	All the planned work that gives confidence the software meets its quality requirements.
SRS	Spesifikasi Keperluan Sistem	The System (or Software) Requirements Specification: what the system must do. KRISA deliverable D03.
STR	Software Trouble Report	The form that records a bug from the moment it is found until it is closed.
UAT	User Acceptance Test	Users try the system to confirm it meets their needs before they accept it.

References and templates for this module

Every line opens the official JDN document at the exact page. Templates download as Word or PDF.

WHERE IT IS IN THE GUIDES

Bab 6, 6.3 Pengurusan Ralat (Rajah 6.2)
page 2 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan

Bab 6, Jadual 6.3 Tahap Severity
page 7 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan

Bab 6, 6.8 Defect Triage dan kriteria keluar UAT
page 21 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan

Bab 5, Jadual 5.5 Ujian Sistem (tiada severity Blocking/Critical)
page 38 · KRISAv2 Bab 5 Fasa Pembangunan

PPrISA 2.0, 5.4.2 i b) Isu: Borang Pelaporan dan Log Penyelesaian Isu
page 92 · PPrISA 2.0 Versi Beta Feb 2025

TEMPLATES TO USE

D11 Laporan Ujian Sistem	WORD	PDF
D14 Laporan Ujian Penerimaan (UAT/PAT)	WORD	PDF
PPrISA14 Borang Pelaporan Isu	WORD	PDF
PPrISA15 Log Penyelesaian Isu	WORD	PDF

From chaos to a controlled quality signal

Daripada kekeliruan kepada isyarat kualiti terkawal

- ✓ Reproduced & logged 5 high-quality defect reports
- ✓ Practiced the full defect lifecycle, including reopen
- ✓ Moved tickets across a live triage board
- ✓ Classified severity independently from priority
- ✓ Traced one bug to its true root cause
- ✓ Computed Defect Density & pass rate from real data

NEXT UP, MODULE 7



Audit, Compliance & Quality Documentation

M06 · W06 · 25 MINUTES · Hari 2, 12:00 hingga 12:25

Hands-On SQA-AI Assistant Workshop

AI bug reports and quality metrics

Turn the failure you reproduced into a complete bug report with KRISAv2 severity, compute metrics from your test run, and prepare a triage summary.

TOOL

opencode 1.18 in the lab folder · model: Big Pickle via OpenCode Zen (free, no API key) · synthetic ShopFast data only

[Step-by-step guide \(web\)](#)[Workshop slides \(W06\)](#)[Setup guide W00](#)[opencode docs](#)

AI bug reports and quality metrics

STEPS

- 1 Run /bug-report with steps, expected and actual result
- 2 Check severity against KRISAv2 Bab 6 Jadual 6.3
- 3 Run /metrics with your run numbers
- 4 Ask for a three-line triage summary

OUTPUT TO SAVE

outputs/m06-bug-report-BUG-CHK-001.md, **outputs/m06-metrics.csv**

TYPE THIS (opencode> in opencode, PS> in PowerShell)

```
opencode> /bug-report Langkah: tambah P002 kuantiti 10 melalui POST /api/cart/items. Dijangka: 201 dan kuantiti 10 diterima (REQ-CHK-01).  
Sebenar: 400 VALIDATION_ERROR. Bukti: ujian tests/ui/m05-senario.spec.js  
gagal, binaan 1.0.0.
```

```
opencode> /metrics kes dirancang 20, dilaksana 18, lulus 13, gagal 5,  
pepijat ditemui 5, pepijat bocor ke produksi 1, saiz kod 1.2 KLOC
```

```
opencode> Ringkaskan pepijat terbuka mengikut severity dan keutamaan untuk  
mesyuarat triage dalam 3 baris. Jangan tulis fail.
```

HUMAN CHECK BEFORE YOU ACCEPT IT

- Steps can be reproduced by someone else
- Severity follows KRISAv2 Jadual 6.3; priority is a business decision
- Metric formulas are shown so the numbers can be rechecked

[Open step-by-step guide](#)

[Module hub](#)

Pembantu AI dalam SQA: AI mendraf, manusia bertanggungjawab

AI assistant in SQA: AI drafts, humans remain accountable

Aliran kerja setiap bengkel Hands-On SQA-AI Assistant Workshop menggunakan opencode

1

Konteks

@SRS, @templat,
AGENTS.md

2

Prompt

arahan /review-srs

3

Draf AI

opencode + model

4

Semak manusia

banding dengan templat
KRISA/PPrISA

5

Simpan bukti

outputs/ + git commit

Guna AI untuk

Draf pelan dan laporan, cadang kes ujian
EP/BVA, jana skrip ujian, ringkaskan log, cari
jurang RTM.

Jangan

Tampal data sebenar agensi, terima output
tanpa semak, biar AI reka ID keperluan, kongsi
sesi awam.

Bukti audit

Simpan prompt, output dan semakan dalam git
supaya jejak audit lengkap (Modul 7).